

Hospital BPMN Logic and Java Code Reference

用途：答辩时解释总图如何调用五张子图、每位成员图中的判断条件，以及哪个 Java 文件让自动节点继续运行。本文按当前 BPMN 与 Java 源码编写。

阅读顺序

先看什么	答案
主流程入口	只启动 Process_Hospital_Master。它依次调用转诊、预约、资金、治疗和信件子流程。
五位成员	邓金旺：转诊；吴家威：预约；吴同宇：资金；林佳强：治疗；肖宇莹：咨询和信件。
Java 启动程序	HospitalExternalWorkers.java 是唯一 main 程序；它一次注册全部六种 job type。
变量传递	五个 Call Activity 都使用 propagateAllChildVariables=true，因此子图判断结果会回到总图。

结论：总图负责跨模块顺序和三种结束结果；每张子图负责本模块的详细分支；Java worker 只处理 BPMN Service Task，不代替 Tasklist 中的人工作业。

1 Total Diagram Logic

总图文件： bpmn/Hospital_Master_Process.bpmn。 流程 ID： Process_Hospital_Master。 总图只有一个启动点和三个结束点。

次序	总图节点	依据变量与结果
1	Referral and Consultant Review	调用 Process_Referral。 referralDecision=ACCEPT 才进入预约； 其他值直接到 Referral Pathway Closed。
2	Appointment and Patient Communication	调用 Process_Appointment。 预约子图结束后才进入资金路径。
3	Funding Payment Refund	调用 Process_Finance。 资金清算网关检查 fundingType、 fundingApproved、 paymentStatus、 urgentTreatmentOverride。
4	Treatment and Chemotherapy	资金通过才调用 Process_Treatment。 治疗子图结束后进入信件模块。
5	Enquiry Clinic Letter Monitoring	调用 Process_EnquiryLetter。 信件子图结束后到 Patient Pathway Completed。

三个结束点

结束事件	到达条件
Referral Pathway Closed	referralDecision 不是 ACCEPT。
Pathway Closed Pending Follow up	资金清算条件不成立。 例如 fundingType=INSURER 且 fundingApproved=false。
Patient Pathway Completed	转诊接受、 资金清算、 治疗完成、 信件子图完成。

总图的关键 FEEL 条件

```
Referral accepted: referralDecision = "ACCEPT"

Financially cleared: fundingType = "HOSPITAL"
or fundingType = "EXEMPTION"
or (fundingType = "INSURER" and fundingApproved = true)
or (fundingType = "PATIENT" and (paymentStatus = "SUCCESSFUL" or urgentTreatmentOverride = true))
```

2 Deng Jinwang Referral and Consultant Review

子图文件：bpmn/23084366_Referral_Consultant_Review/BPMN/Referral_Consultant_Review.bpmn。流程 ID：Process_Referral。这个模块没有 Service Task，因此 ReferralWorkers.java 只明确说明“不注册 worker”。

节点	判断变量	走向
Record Referral and Supporting Documents	informationComplete、urgentReferral	资料进入检查；紧急信息会触发临床升级路径。
Check Supporting Information	资料是否完整	不完整时回到资料补充。
Perform Consultant Review	referralDecision	REJECT：转诊关闭；MORE_INFORMATION：补资料后再审；ACCEPT：返回总图预约；REDIRECT：转往其他路径。

与总图的交接：最关键输出是 referralDecision。总图不读取任何转诊任务名称，只读取这个变量。

3 Wu Jiawei Appointment and Patient Communication

子图文件:

bpmn/23084364_Appointment_Patient_Communication/BPMN/Appointment_Patient_Communication.bpmn。

流程 ID: Process_Appointment。

节点	变量或自动任务	逻辑
Receive Booking Request	requestedSlotAvailable	提交预约信息后，进入 search-appointment-slot。
Search External Scheduling Service	Java job: search-appointment-slot	AppointmentWorkers 将 requestedSlotAvailable 转成 slotAvailable，并计算 appointmentWithinTwoWeeks。
Suitable Slot Available?	slotAvailable	true：确认预约；false：记录无合适时段、升级或重新搜索。
Appointment Within Two Weeks?	appointmentWithinTwoWeeks	true：直接发送预约信；false：电话联系病人。
Telephone Outcome	contactOutcome	ANSWERED 完成沟通；RESCHEDULE 回到搜号源；WRONG_NUMBER 记录错误号码。
Send Appointment Letter	Java job: send-appointment-letter	worker 写入 appointmentLetterSent=true；子图完成，回到总图资金模块。

4 Wu Tongyu Funding Payment and Refund

子图文件: bpmn/23084362_Funding_Payment_Refund/BPMN/Funding_Payment_Refund.bpmn。流程 ID: Process_Finance。

节点	变量或自动任务	逻辑
Funding Type	fundingType	HOSPITAL / EXEMPTION: 直接清算; INSURER: 等待批准; PATIENT: 调用支付服务。
Insurer Funding Approved?	fundingApproved	true: 清算; false: Funding Pending, 回到总图后进入 Pending Follow up 结束。
Send Secure Payment Request	Java job: process-payment	FinanceWorkers 读取 paymentScenario, 并写入 paymentStatus。
Payment Status	paymentStatus	SUCCESSFUL: 清算; DECLINED / CANCELLED / INCOMPLETE: 重试; CONFIRMATION_LOST / DUPLICATE_RISK: 调查后判断紧急覆盖。
Urgent Treatment Despite Unconfirmed Payment?	urgentTreatmentOverride	true: 记录临床覆盖; false: 保持未清算。
Refund Approved?	refundRequired、refundApproved	同时为 true 时调用 process-refund; 否则退款不执行。

5 Lin Jiaqiang Treatment and Chemotherapy

子图文件: bpmn/23084368_Treatment_Chemotherapy/BPMN/Treatment_Chemotherapy.bpmn。流程 ID: Process_Treatment。

节点	变量或自动任务	逻辑
Clinical Assessment	patientConsent	true: 记录同意并创建治疗请求; false: 治疗不继续。
Treatment Request	treatmentAuthorised	true: 检查资源; false: 退回并重新授权。
Check Treatment Resources	Java job: check-treatment-resource	TreatmentWorkers 将 requestedTreatmentAvailable 写成 treatmentAvailable。
Resources Available?	treatmentAvailable	true: 预约治疗; false: 保持待处理、更新资源后重试。
Treatment Cycle Review	moreCycles	true: 进入血检和临床复审; false: Treatment Complete Clinic Letter Handover。
Clinical Decision	clinicalDecision	CONTINUE: 下一周期; DELAY: 延迟后复审; MODIFY: 修改治疗并判断 financialImpact。

与总图的交接: 治疗子图无论“完成治疗”或“临床不继续”都能结束, 然后总图调用信件模块; 总图没有额外读取治疗变量。

6 Xiao Yuying Enquiry Clinic Letter Monitoring

子图文件： bpmn/24000813_Enquiry_ClinicLetter_Monitoring/BPMN/Enquiry_ClinicLetter_Monitoring.bpmn。

流程 ID： Process_EnquiryLetter。

节点	变量或自动任务	逻辑
Enquiry Type	enquiryType	ADMINISTRATIVE、FINANCIAL、CLINICAL 分别交给相应人员；urgentClinicalConcern=true 会立即升级。
Prepare Clinic Letter	clinicLetterWithin7Days	true：进入临床内容审批；false：加入监控并进入每周提醒。
Clinical Content Approved?	clinicLetterApproved	true：行政核对收件人；false：重新判断是否仍可在七天内发出。
Outstanding Duration	letterAgeBand	7D_TO_1M：提醒顾问；OVER_1M：升级行政经理；OVER_3M：升级高层。
Clinic Letter Completed?	clinicLetterCompleted	true：回到审批；false：继续每周监控循环。
Distribute Clinic Letter	Java job: send-clinic-letter	worker 要求 clinicLetterApproved=true；完成后写入 clinicLetterSent=true。

7 Java Program and Worker Registration

启动程序：External_Workers/src/main/java/org/hospital/workers/HospitalExternalWorkers.java。main 方法使用 ZEEBE_ADDRESS（默认 127.0.0.1:26500）创建客户端，然后注册所有模块 worker，并保持进程运行。

Java 文件	负责的 BPMN job type	关键输出
AppointmentWorkers.java	search-appointment-slot; send-appointment-letter	slotAvailable; appointmentWithinTwoWeeks; appointmentLetterSent
TreatmentWorkers.java	check-treatment-resource	treatmentAvailable
FinanceWorkers.java	process-payment; process-refund	paymentStatus; refundStatus
EnquiryClinicLetterWorkers.java	send-clinic-letter	clinicLetterSent
ReferralWorkers.java	无	转诊图没有 Service Task，因此不订阅 job。
WorkerSupport.java	所有 job 共用	注册、完成 job、失败重试、变量读写工具。

运行关系：Tasklist 完成人工任务 → BPMN 到达 Service Task → 对应 Java worker 接收 job → worker 写回变量 → 网关依据变量决定下一条线。

Appendix A Full Java Source Files

以下是当前项目中的完整 Java worker 源码。HospitalExternalWorkers.java 是要启动的主程序。

HospitalExternalWorkers.java

```
package org.hospital.workers;

import io.camunda.zeebe.client.ZeebeClient;
import io.camunda.zeebe.client.api.worker.JobWorker;

import java.util.ArrayList;
import java.util.List;

public final class HospitalExternalWorkers {

    private static final String ADDRESS =
        System.getenv().getOrDefault("ZEEBE_ADDRESS", "127.0.0.1:26500");

    private HospitalExternalWorkers() {}

    public static void main(String[] args) throws InterruptedException {
        System.out.println("Connecting to Zeebe at " + ADDRESS);

        try (ZeebeClient client = ZeebeClient.newClientBuilder()
            .gatewayAddress(ADDRESS)
            .usePlaintext()
            .build()) {

            List<JobWorker> workers = new ArrayList<>();

            ReferralWorkers.register(client, workers);
            AppointmentWorkers.register(client, workers);
            TreatmentWorkers.register(client, workers);
            FinanceWorkers.register(client, workers);
            EnquiryClinicLetterWorkers.register(client, workers);

            System.out.println("Workers are ready.");
            System.out.println("Registered worker count: " + workers.size());
            System.out.println("Expected job types:");
            System.out.println(" - search-appointment-slot");
            System.out.println(" - send-appointment-letter");
            System.out.println(" - check-treatment-resource");
            System.out.println(" - process-payment");
            System.out.println(" - process-refund");
            System.out.println(" - send-clinic-letter");

            Thread.currentThread().join();

        }
    }
}
```

WorkerSupport.java

```
package org.hospital.workers;

import io.camunda.zeebe.client.ZeebeClient;
import io.camunda.zeebe.client.api.response.ActivatedJob;
import io.camunda.zeebe.client.api.worker.JobClient;
import io.camunda.zeebe.client.api.worker.JobWorker;

import java.time.Duration;
import java.time.LocalDate;
import java.time.format.DateTimeParseException;
import java.util.HashMap;
import java.util.Map;
import java.util.UUID;
import java.util.function.Function;

public final class WorkerSupport {

    private WorkerSupport() {}

    public static JobWorker open(
        ZeebeClient client,
        String jobType,
        Function<Map<String, Object>, Map<String, Object>> handler) {

        return client.newWorker()
            .jobType(jobType)
            .handler((jobClient, job) -> complete(jobClient, job, jobType, handler))
    }
}
```

```

        .name("hospital-demo-" + jobType)
        .timeout(Duration.ofSeconds(30))
        .open();
    }

    private static void complete(
        JobClient jobClient,
        ActivatedJob job,
        String jobType,
        Function<Map<String, Object>, Map<String, Object>> handler) {

        try {
            Map<String, Object> input = job.getVariablesAsMap();
            Map<String, Object> output = handler.apply(input);

            jobClient.newCompleteCommand(job)
                .variables(output)
                .send()
                .join();

            System.out.println("[COMPLETED] " + jobType
                + " key=" + job.getKey()
                + " output=" + output);

        } catch (Exception exception) {
            String message = exception.getMessage() == null
                ? exception.getClass().getSimpleName()
                : exception.getMessage();

            jobClient.newFailCommand(job)
                .retries(Math.max(0, job.getRetries() - 1))
                .errorMessage(message)
                .send()
                .join();

            System.err.println("[FAILED] " + jobType
                + " key=" + job.getKey()
                + " error=" + message);
        }
    }

    public static boolean bool(Object value, boolean defaultValue) {
        if (value == null) return defaultValue;
        if (value instanceof Boolean b) return b;
        String s = String.valueOf(value).trim();
        if (s.isEmpty()) return defaultValue;
        return Boolean.parseBoolean(s);
    }

    public static String str(Object value, String defaultValue) {
        if (value == null) return defaultValue;
        String s = String.valueOf(value).trim();
        return s.isEmpty() ? defaultValue : s;
    }

    public static LocalDate parseDate(Object value) {
        String raw = str(value, "");
        if (raw.isEmpty()) return null;

        String datePart = raw.length() >= 10 ? raw.substring(0, 10) : raw;
        try {
            return LocalDate.parse(datePart);
        } catch (DateTimeParseException ex) {
            return null;
        }
    }

    public static String shortId(String prefix) {
        return prefix + "-" + UUID.randomUUID()
            .toString()
            .replace("-", "")
            .substring(0, 8)
            .toUpperCase();
    }

    public static Map<String, Object> output() {
        return new HashMap<>();
    }
}

```

ReferralWorkers.java

```
package org.hospital.workers;

import io.camunda.zeebe.client.ZeebeClient;
import io.camunda.zeebe.client.api.worker.JobWorker;

import java.util.List;

/**
 * Corresponds to:
 * Referral_Consultant_Review/BPMN/Referral_Consultant_Review.bpmn
 *
 * This BPMN currently contains User Tasks, Gateways, Send/Receive Tasks and
 * Message Flows, but no BPMN Service Task with a Zeebe job type.
 *
 * Therefore there is intentionally NO external Job Worker subscription here.
 * The file is kept so every BPMN area has a matching Java source file and the
 * absence of a Referral worker is explicit rather than accidental.
 */
public final class ReferralWorkers {

    private ReferralWorkers() {}

    public static void register(ZeebeClient client, List<JobWorker> workers) {
        System.out.println("[INFO] Referral BPMN has no external Service Task job type; no worker registration required.");
    }
}
```

AppointmentWorkers.java

```
package org.hospital.workers;

import io.camunda.zeebe.client.ZeebeClient;
import io.camunda.zeebe.client.api.worker.JobWorker;

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.List;
import java.util.Map;

public final class AppointmentWorkers {

    public static final String SEARCH_SLOT = "search-appointment-slot";
    public static final String SEND_APPOINTMENT_LETTER = "send-appointment-letter";

    private AppointmentWorkers() {}

    public static void register(ZeebeClient client, List<JobWorker> workers) {
        workers.add(WorkerSupport.open(client, SEARCH_SLOT, AppointmentWorkers::searchAppointmentSlot));
        workers.add(WorkerSupport.open(client, SEND_APPOINTMENT_LETTER, AppointmentWorkers::sendAppointmentLetter));
    }

    public static Map<String, Object> searchAppointmentSlot(Map<String, Object> input) {
        Map<String, Object> out = WorkerSupport.output();

        boolean available = WorkerSupport.bool(input.get("requestedSlotAvailable"), true);
        out.put("slotAvailable", available);

        if (!available) {
            out.put("appointmentWithinTwoWeeks", false);
            out.put("schedulingStatus", "NO_SUITABLE_SLOT");
            return out;
        }

        LocalDate appointmentDate = WorkerSupport.parseDate(input.get("requestedAppointmentDate"));
        if (appointmentDate == null) {
            appointmentDate = LocalDate.now().plusDays(10);
        }

        long days = ChronoUnit.DAYS.between(LocalDate.now(), appointmentDate);
        boolean withinTwoWeeks = days >= 0 && days <= 14;

        out.put("appointmentDate", appointmentDate.toString());
        out.put("appointmentWithinTwoWeeks", withinTwoWeeks);
        out.put("schedulingStatus", "SLOT_FOUND");
        out.put("schedulingReference", WorkerSupport.shortId("SCH"));

        return out;
    }

    public static Map<String, Object> sendAppointmentLetter(Map<String, Object> input) {
        Map<String, Object> out = WorkerSupport.output();
    }
}
```

```

        out.put("appointmentLetterSent", true);
        out.put("appointmentCorrespondenceReference", WorkerSupport.shortId("APT"));
        out.put("appointmentLetterSentDate", LocalDate.now().toString());

        return out;
    }
}

```

TreatmentWorkers.java

```

package org.hospital.workers;

import io.camunda.zeebe.client.ZeebeClient;
import io.camunda.zeebe.client.api.worker.JobWorker;

import java.time.LocalDate;
import java.util.List;
import java.util.Map;

public final class TreatmentWorkers {

    public static final String CHECK_RESOURCE = "check-treatment-resource";

    private TreatmentWorkers() {}

    public static void register(ZeebeClient client, List<JobWorker> workers) {
        workers.add(WorkerSupport.open(client, CHECK_RESOURCE, TreatmentWorkers::checkTreatmentResource));
    }

    public static Map<String, Object> checkTreatmentResource(Map<String, Object> input) {
        Map<String, Object> out = WorkerSupport.output();

        boolean available = WorkerSupport.bool(input.get("requestedTreatmentAvailable"), true);

        out.put("treatmentAvailable", available);
        out.put("resourceCheckReference", WorkerSupport.shortId("RES"));
        out.put("resourceCheckDate", LocalDate.now().toString());
        out.put("resourceCheckStatus", available ? "AVAILABLE" : "TEMPORARILY_UNAVAILABLE");

        return out;
    }
}

```

FinanceWorkers.java

```

package org.hospital.workers;

import io.camunda.zeebe.client.ZeebeClient;
import io.camunda.zeebe.client.api.worker.JobWorker;

import java.time.LocalDate;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Set;

public final class FinanceWorkers {

    public static final String PROCESS_PAYMENT = "process-payment";
    public static final String PROCESS_REFUND = "process-refund";

    private static final Set<String> ALLOWED_PAYMENT_SCENARIOS = Set.of(
        "SUCCESSFUL",
        "DECLINED",
        "CANCELLED",
        "INCOMPLETE",
        "CONFIRMATION_LOST",
        "DUPLICATE_RISK"
    );

    private FinanceWorkers() {}

    public static void register(ZeebeClient client, List<JobWorker> workers) {
        workers.add(WorkerSupport.open(client, PROCESS_PAYMENT, FinanceWorkers::processPayment));
        workers.add(WorkerSupport.open(client, PROCESS_REFUND, FinanceWorkers::processRefund));
    }

    public static Map<String, Object> processPayment(Map<String, Object> input) {
        Map<String, Object> out = WorkerSupport.output();

        String scenario = WorkerSupport.str(input.get("paymentScenario"), "SUCCESSFUL")
    }
}

```

```

        .toUpperCase(Locale.ROOT);

        if (!ALLOWED_PAYMENT_SCENARIOS.contains(scenario)) {
            throw new IllegalArgumentException(
                "Unsupported paymentScenario: " + scenario
                + ". Expected SUCCESSFUL, DECLINED, CANCELLED, INCOMPLETE, CONFIRMATION_LOST or DUPLICATE_RISK.");
        }

        out.put("paymentStatus", scenario);
        out.put("paymentDate", LocalDate.now().toString());
        out.put("paymentProvider", "SIMULATED_EXTERNAL_PAYMENT_SERVICE");

        String approvedAmount = WorkerSupport.str(input.get("approvedAmount"), "");
        if (!approvedAmount.isEmpty()) {
            out.put("paymentAmount", approvedAmount);
        }

        if ("SUCCESSFUL".equals(scenario)) {
            out.put("transactionReference", WorkerSupport.shortId("PAY"));
            out.put("paymentInvestigationRequired", false);
        } else if ("CONFIRMATION_LOST".equals(scenario) || "DUPLICATE_RISK".equals(scenario)) {
            out.put("paymentInvestigationRequired", true);
        } else {
            out.put("paymentInvestigationRequired", false);
        }

        return out;
    }

    public static Map<String, Object> processRefund(Map<String, Object> input) {
        Map<String, Object> out = WorkerSupport.output();

        boolean refundApproved = WorkerSupport.bool(input.get("refundApproved"), false);
        boolean refundRequired = WorkerSupport.bool(input.get("refundRequired"), refundApproved);

        if (!refundRequired) {
            out.put("refundStatus", "NOT_REQUIRED");
            return out;
        }

        if (!refundApproved) {
            out.put("refundStatus", "NOT_APPROVED");
            return out;
        }

        out.put("refundStatus", "APPROVED_AND_SENT");
        out.put("refundReference", WorkerSupport.shortId("REF"));
        out.put("refundDate", LocalDate.now().toString());

        String refundAmount = WorkerSupport.str(input.get("refundAmount"), "");
        if (!refundAmount.isEmpty()) {
            out.put("processedRefundAmount", refundAmount);
        }

        return out;
    }
}

```

EnquiryClinicLetterWorkers.java

```

package org.hospital.workers;

import io.camunda.zeebe.client.ZeebeClient;
import io.camunda.zeebe.client.api.worker.JobWorker;

import java.time.LocalDate;
import java.util.List;
import java.util.Map;

public final class EnquiryClinicLetterWorkers {

    public static final String SEND_CLINIC_LETTER = "send-clinic-letter";

    private EnquiryClinicLetterWorkers() {}

    public static void register(ZeebeClient client, List<JobWorker> workers) {
        workers.add(WorkerSupport.open(client, SEND_CLINIC_LETTER, EnquiryClinicLetterWorkers::sendClinicLetter));
    }

    public static Map<String, Object> sendClinicLetter(Map<String, Object> input) {
        boolean approved = WorkerSupport.bool(input.get("clinicLetterApproved"), false);
        if (!approved) {
            throw new IllegalArgumentException(

```

```
        "Clinic Letter cannot be distributed because clinicLetterApproved is not true.");
    }

    Map<String, Object> out = WorkerSupport.output();
    out.put("clinicLetterSent", true);
    out.put("clinicLetterDistributionReference", WorkerSupport.shortId("CL"));
    out.put("clinicLetterDistributionDate", LocalDate.now().toString());

    return out;
}
}
```